1. The Traditional Switch Statement (Legacy / Pre-Java 14)

The traditional switch evaluates a variable and jumps to the matching case. It relies heavily on the break keyword to stop execution.

Syntax & Example:

Java

```
package com.kodelix.clearflow;

public class DocumentProcessor {

    public void processDocument(String docType) {
        // Traditional Switch Statement
        switch (docType) {
            case "INVOICE":
                System.out.println("Extracting pricing and tax data.");
                break; // CRITICAL: Stops execution from falling to the next case
            case "ID_CARD":
            case "PASSPORT": // Stacking cases: Both execute the code below
                System.out.println("Extracting identity and photo data.");
                break;
            case "RECEIPT":
                System.out.println("Extracting total amount.");
                break;
            default:
                System.out.println("Unknown document type. Manual review required.");
                // No break needed here as it is the last statement
        }
    }
}
```

The Vulnerability (Fall-Through):

If you forget to write break;, Java does not throw an error. Instead, it executes the matching case **and every single case below it** until it hits a break or the end of the switch. This is called "Fall-Through" and is the source of countless critical bugs in legacy code.

2. The Modern Switch Expression (Java 14+)

Java 14 revolutionized the switch. It introduced the arrow -> syntax, eliminated the need for break, and allowed the switch to directly return (yield) a value into a variable.

Syntax & Example:

Java

```
package com.kodelix.clearflow;

public class DocumentAnalyzer {

    public String determineProcessingEngine(String docType) {

        // Modern Switch Expression assigning a result directly to a variable
```

```
        String engine = switch (docType) {
            case "INVOICE" -> "FinancialOCR_v2";
            case "ID_CARD", "PASSPORT" -> "IdentityScanner_v1"; // Comma-separated cases
            case "RECEIPT" -> "BasicTextExtractor";
            default -> "ManualReviewQueue";
        };

        return engine;
    }
}
```

Key Advantages:

1. **No Fall-Through:** The -> syntax executes *only* the code on the right side of the arrow. You can never accidentally fall through to the next case.
2. **Exhaustiveness:** If you are using a Switch Expression to assign a variable, the compiler forces you to include a default case to guarantee that a value is always returned.
3. **Multiple Labels:** You can group cases cleanly using commas (e.g., case "ID_CARD", "PASSPORT" -
>).

3. The yield Keyword (Multi-line Modern Switch)

If the logic inside a modern switch case requires multiple lines of code before returning a value, you must use curly braces {} and the yield keyword instead of return.

Java

```
int confidenceScore = 85;

String status = switch (confidenceScore) {
    case 100 -> "Perfect Match";
    case 90, 85 -> {
        System.out.println("Logging slight discrepancy...");
        // 'yield' acts like 'return' specifically for the switch expression
        yield "Acceptable Match";
    }
    default -> "Re-scan Required";
};
```

Interview Vault: Tricky & Viral Questions

> **Q: "What data types are allowed to be evaluated in a switch statement?"**
>
> **Answer:** > * **Allowed:** byte, short, char, int, String, and Enums. (And their respective wrapper classes like Integer or Character).
>
> - **NOT Allowed:** long, float, double, and boolean.
> *(Note: This is a highly common trick question. If an interviewer gives you a* switch (myLongVariable)*, it will fail to compile).*
>
> **Q: "What will the following code print?"**
>
> Java

```
int accessLevel = 2;
switch (accessLevel) {
    case 1:
        System.out.print("User ");
    case 2:
        System.out.print("Admin ");
    case 3:
        System.out.print("SuperAdmin ");
    default:
        System.out.print("Guest ");
}
```

Answer: It will print: **Admin SuperAdmin Guest**

The Trap: This tests your knowledge of "Fall-Through." Because there are no break statements, execution starts at case 2 and uncontrollably bleeds into case 3 and default.

Q: "What happens if you pass null into a switch statement, like switch(myString) where myString is null?"

Answer: It throws a NullPointerException immediately at runtime. A switch evaluates the value of the variable, and you cannot evaluate the value of null. *(Bonus points: Mention that starting in Java 21, you can explicitly write* case null -> *to handle this safely, but historically it results in a crash).*